



Σχεδιασμός Ενσωματωμένων Συστημάτων

9ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

3η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

Εισαγωγή	2
Άσκηση 1	2
Ερώτημα Α	2
Ερώτημα Β	3
Ερώτημα Γ	4
Ερώτημα Δ	10
Άσκηση 2	12
Άσκηση 3	15
Σημειώσεις για τους κώδικες	17

Εισαγωγή

Στην άσκηση αυτή θα εξοικειωθούμε στον προγραμματισμό Field Programmable Gate Arrays (FPGA) με High Level Synthesis (HLS). Η άσκηση αυτή περιλαμβάνει τον κώδικα για ένα σύστημα προτάσεων (Recommendation System - RS) για ταινίες, το οποίο βασίζεται στον ταξινομητή K κοντινότερων γειτόνων και χρησιμοποιεί ένα υποσύνολο του MovieLens dataset. Ο στόχος μας είναι να πραγματοποιήσουμε HLS optimizations για να μεγιστοποιήσουμε το speedup της εκτέλεσης του υπολογισμού των ευκλείδειων αποστάσεων για τον αλγόριθμο σε hardware και software.

Άσκηση 1

Ερώτημα A

Θέτουμε την συνάρτηση calcDistancesHW ως hardware function και για το Estimated Performance παρουσιάζουμε παρακάτω τα αποτελέσματα μας χωρίς κάποιο optimization:

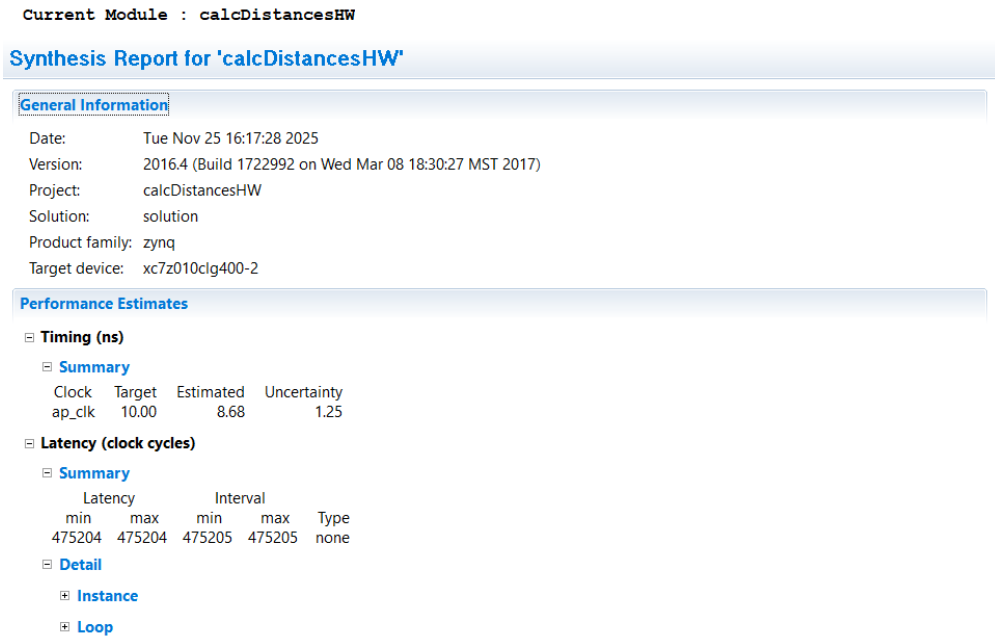


Figure 1: Performance without optimization

Details

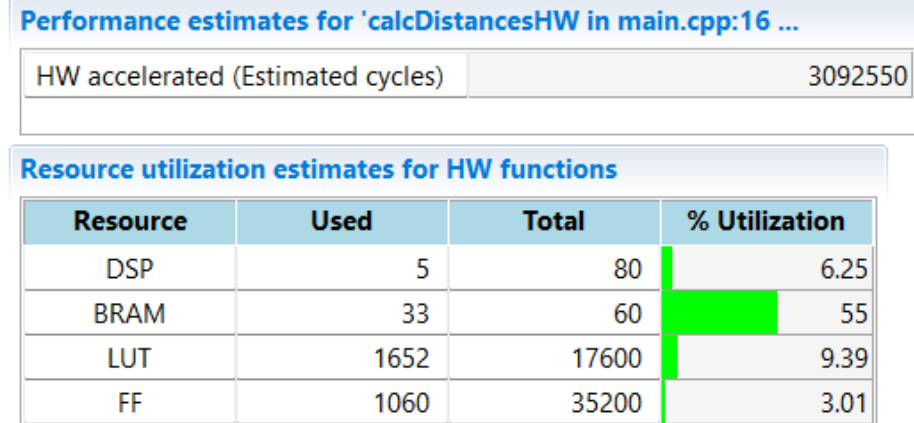


Figure 2: Utilization without optimization

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- LOAD_DATA_HW_TMP	34816	34816	34	-	-	1024	no
+ LOAD_DATA_HW_TMP.1	32	32	1	-	-	32	no
- LOAD_MOVIE_TMP	64	64	2	-	-	32	no
- COMPUTE_DISTS	438272	438272	428	-	-	1024	no
+ COMPUTE_DISTS.1	416	416	13	-	-	32	no
- WRITE_DISTS	2048	2048	2	-	-	1024	no

Figure 3: Loop Estimates without optimization

Από τα παραπάνω διαγράμματα μπορούμε να παρατηρήσουμε χρονικά estimations για τον κύκλο του ρολογιού για το design μας αν τρέξει σε fpga, μπορούμε να καταγράψουμε τους κύκλους που αναμένεται να πραγματοποιήσει η HW accelerated εκδοση του design μας (3092550) και την χρήση σε πόρους του fpga (DSP, BRAM, LUT, FF) τα οποία θα χρησιμοποιήσουμε ως σημείο αναφοράς για την συνέχεια της ανάλυσής μας. Τέλος, μπορούμε να παρατηρήσουμε και στοιχεία για την εκτέλεση των loop, για τα οποία παρατηρούμε ότι το χρονικά πιο ακριβό είναι το COMPUTE_DISTS με 438272 κύκλους.

Ερώτημα Β

Το επόμενο βήμα μας είναι χωρίς ακόμη optimizations να παράξουμε το bitstream για το design μας και με την sd του zybo να το περάσουμε σε αυτό και να δοκιμάσουμε να το τρέξουμε. Η εφαρμογή τρέχει στο zybo με τα παρακάτω αποτελέσματα:

```

No lease, forking to background
done.
Starting Dropbear SSH server: Generating key, this may take a while...
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDP88miWfmlYwkBpnZOQD5tg56WeU9SLyam8sOv+uWI
Zl14n+7urB996ydE3mtcPslJEpC7HrzNS0HCujJfHPBgchab2BriPlpAtgOdVnHgNiBlBgYTIWIney+
kEJbGAFfFLfF42ofx7Dco6DKScjju/RVQ8n5pmHAuaah+WojJ1mxQ9dZgnlMgVyltwvIltvmFL9iLF4e
PBIO1c0tYcK62HeZz1Gla2cms6TtkKSUg5idZ02gIfpgzRt+5ug6iRcAHg/fwtUX7gDMpHLpgo7cjNO
uGUnFEiEKpI5iEm6TNHvgzAy0FkqtJmoeXN/K0WxLiED4+GkmalOFTQ43Tt9 root@Avnet-Digilent
-ZedBoard-2016_3
Fingerprint: md5 65:ea:c0:be:23:ca:51:8e:00:25:3a:03:3b:1d:0b:b5
dropbear.
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting tcf-agent: OK

sh-4.3# ls
bin      dev      home     install  media    proc      sbin     tmp      var
boot     etc      init     lib      mnt      run       sys      usr
sh-4.3# cd /mnt
sh-4.3# ls
BOOT.BIN          System Volume Information  image.ub
README.txt        _sds                      zybo.elf
sh-4.3# ./zybo.elf
KNN based Recommendation System Acceleration
Input movie id = 0
Started distance calculations on software...
Finished distance calculations on software...
Started distance calculations on hardware...
Finished distance calculations on hardware...

Recommendation system start to make inference
...
Recommendations for movie with id 0:
0. Jurassic Park (1993), with distance of 10.7121
1. Fish Called Wanda A (1988), with distance of 11.0793
2. Back to the Future (1985), with distance of 11.6404
3. Star Wars: Episode VI - Return of the Jedi (1983), with distance of 11.8849
4. Lion King The (1994), with distance of 12.0623
5. Raising Arizona (1987), with distance of 12.1552
6. Wizard of Oz The (1939), with distance of 12.1861
7. Batman (1989), with distance of 12.2066
8. Princess Bride The (1987), with distance of 12.3085
9. E.T. the Extra-Terrestrial (1982), with distance of 12.3996

Hardware cycles : 3092853
Software cycles : 1582622
Speedup         : 0.511703
Correct = 1024, Score = 1.000000
sh-4.3#

```

Figure 4: Unoptimized run in zybo

Παρατηρούμε ότι δεν έχουμε speedup σε αυτήν την περίπτωση. Οι κύκλοι του hardware δεν είναι ακριβώς ίδιοι με το estimation, όμως κρίνεται ότι το estimation είναι ένα αρκετά καλό μέτρο σύγκρισης για τα πειράματά μας στην συνέχεια, αφού παρέχει μια αρκετά καλή προσέγγιση των πραγματικών κύκλων της εφαρμογής.

Ερώτημα Γ

Στην συνέχεια πραγματοποιούμε optimizations για να βελτιώσουμε την απόδοση της εφαρμογής μας και αρχικά πραγματοποιούμε performance estimations μέχρι να πετύχουμε ένα αποτέλεσμα που να φαίνεται αρκετά ικανοποιητικό για να δοκιμαστεί στο zybo. Παρακάτω παρουσιάζουμε συνοπτικά τους πιο σημαντικούς συνδυασμούς directives που δοκιμάσαμε:

- Pipelining

Το πρώτο μας βήμα είναι να χρησιμοποιήσουμε το:

```
# pragma HLS pipeline II = 1
```

προσπαθώντας να πετύχουμε pipelined for loops. Σημειώνεται ότι στα διπλά loops αρχικά δοκιμάσαμε pipeline στο εσωτερικό loop (η προσέγγισή μας αλλάζει και εξηγείται παρακάτω).

Τα αποτελέσματα που πετυχαίνουμε παρουσιάζονται παρακάτω:

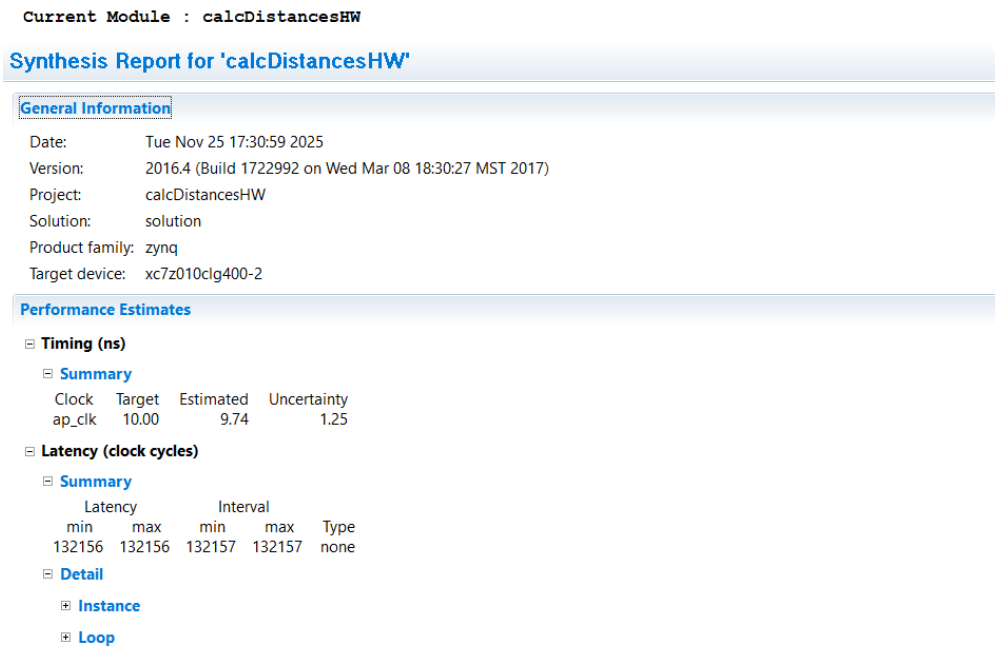


Figure 5: Performance with pipelining

Details

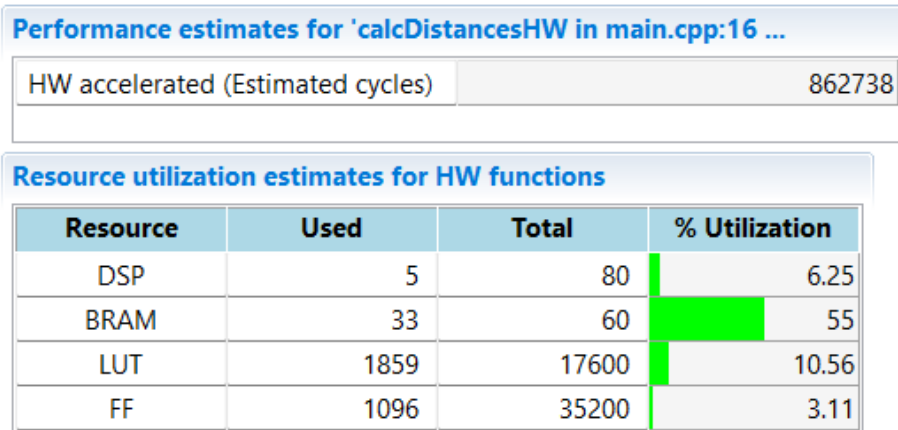


Figure 6: Utilization with pipelining

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- LOAD_DATA_HW_TMP_L	32768	32768	2	1	1	32768	yes
- LOAD_MOVIE_TMP	32	32	2	1	1	32	yes
- COMPUTE_DISTIS_L	98324	98324	24	3	1	32768	yes
- WRITE_DISTIS	1024	1024	2	1	1	1024	yes

Figure 7: Loop Estimates with pipelining

Παρατηρούμε μια μικρή επιβάρυνση στην εκτίμηση για τον χρόνο κύκλου ρολογιού, εντός των ορίων όμως. Οι κύκλοι εκτέλεσης της εφαρμογής μας παρουσιάζουν σημαντική εκτιμώμενη μείωση στους 862738 με μια αύξηση όπως είναι αναμενόμενο σε LUTs και FFs. Για τα loops παρατηρούμε μείωση της καθυστερήσής τους με το pipelined designed και όλα για όλα τα loops επιτυγχάνεται πλήρες pipelining ($\Pi = 1$), εκτός από το COMPUTE_DISTIS που έχει $\Pi = 3$. Σε αυτήν την περίπτωση επειδή στο εσωτερικό loop ο υπολογισμός του sum σε κάθε επανάληψη εξαρτάται από την τιμή του sum στην προηγούμενη επανάληψη δεν μπορούμε να πετύχουμε καλύτερο pipelining (στην συνέχεια παρουσιάζεται μια εναλλακτική, καλύτερη προσέγγιση μας).

- Loop unrolling

Το loop LOAD_MOVIE_TMP εκτελεί 32 επαναλήψεις και επιχειρούμε αντί για pipeline να το κάνουμε unroll με:

```
#pragma HLS unroll
```

Προσέχουμε, φυσικά ο πίνακας που αφορά το loop αυτό και έχει διάσταση 32 να είναι complete partition, εφόσον δεν παρατηρούμε μεγάλο πρόβλημα στην κατανάλωση πόρων. Ο σκοπός που το πραγματοποιούμε είναι για να καταφέρουμε να έχουμε στην πραγματικότητα παράλληλα readings.

Έχουμε:

Details

Performance estimates for 'calcDistancesHW in main.cpp:16 ...

HW accelerated (Estimated cycles)	862569
-----------------------------------	--------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	5	80	6.25
BRAM	33	60	55
LUT	2232	17600	12.68
FF	2111	35200	6

Figure 8: Utilization with loop unrolling

Παρατηρούμε μια μικρή μείωση σε estimated cycles με αύξηση σε LUTs και FFs.

- Κάποιες αποτυχημένες προσπάθειες βελτίωσης που είχαμε πριν το επομενό μας βήμα ήταν οι προσπάθειες για partition των πινάκων με χρήση:

```
#pragma HLS ARRAY_PARTITION
```

Αρχικά, στην προσπάθεια μας για partition δοκιμάσαμε για τον πίνακα data_hw_tmp να κάνουμε block partition και είτε block είτε cyclic partition στην 1η διάσταση. Παρατηρούμε ότι δεν πετυχαίνουμε σημαντική βελτίωση και προχωράμε σε ανάλυση για array partitions με cyclic partition στην διάσταση 2 (παρακάτω εξηγούμε και γιατί η τελική επιλογή είναι καλύτερη).

- Pipeline / cyclic partition - dimension 2

Το επόμενο βήμα που παρουσιάζει βελτίωση είναι η μετακίνηση του pipelining του loop COMPUTE_DISTS στο εξωτερικό loop και η αλλαγή του partition του πίνακα data_hw_tmp σε cyclic με factor 8 αρχικά. Παρατηρούμε, όμως ότι για αυτόν τον συνδυασμό directives η χρήση σε πόρους του zybo δεν επαρκεί και έτσι πρέπει να περιοριστούμε σε factor 4 σε 1η προσέγγιση (σημειώνουμε σαν σκέψη την πιθανή βελτίωση με αύξηση του array partition factor, καθώς στην συνέχεια αναλύεται όταν είναι εφικτό). Σε επόμενο μας βήμα θα εκμεταλλευτούμε αυτό το partition που πραγματοποιήσαμε.

Details

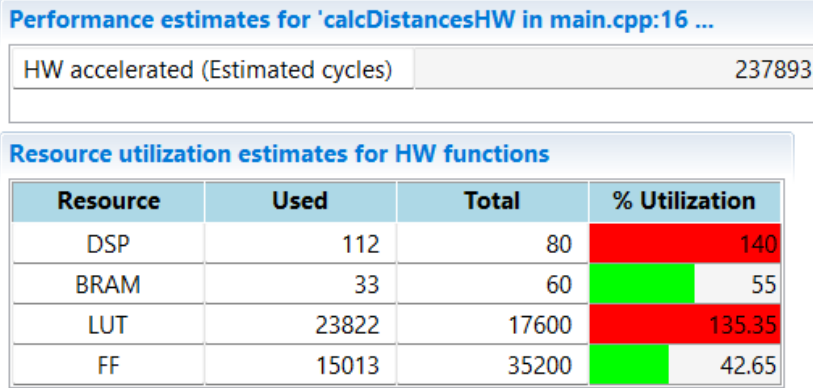


Figure 9: Utilization pipeline-partition with factor 8

Details

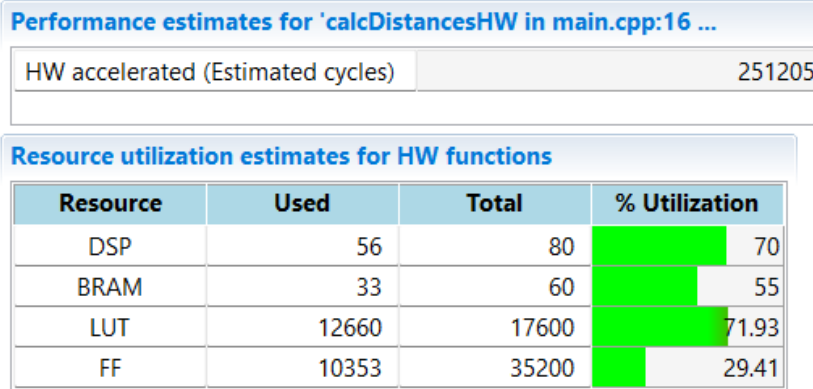


Figure 10: Utilization pipeline-partition with factor 4

Για την συμμετοχή του array partition στην βελτίωση παρατηρήσαμε ότι για ίδιο partition στην 2η διάσταση έχουμε καλύτερα αποτελέσματα και αυτό συμβαίνει γιατί το βαρύτερο υπολογιστικά κομμάτι στα loops που αφορά τον πίνακα των δεδομένων ακολουθεί προσπελάσεις στην 2η διάσταση. Επιπλέον,

η cyclic μέθοδος φαίνεται να κυριαρχεί έναντι της block προσέγγισης, γιατί με την κυκλική αναθέτει στα banks με κυκλικό τρόπο τα δεδομένα (θέσεις του πίνακα ουσιαστικά), ενώ το block σε συνεχόμενα κομμάτια και αυτό έχει ως αποτέλεσμα να μπορεί να γίνει καλύτερη παραλληλοποίηση με την κυκλική προσέγγιση.

Δοκιμάσαμε την προσέγγιση αυτή για παράγοντα partition 2,4 και 8 και παρατηρούμε ότι για 8 έχουμε υπέρβαση των διαθέσιμων πόρων, για 4 έχουμε αρκετά καλή χρήση των πόρων, αν και χειρότερη επίδοση και για παράγοντα 2 έχουμε 277803 κύκλους και χαμηλότερη χρήση πόρων. Θεωρούμε ότι η βέλτιστη επιλογή μας είναι $\text{factor} = 4$.

- Double pipelining moved

Επειδή στην προηγούμενη περίπτωση παρατηρήσαμε ότι η μετακίνηση της εντολής για pipelining στο εξωτερικό loop είχε σημαντική βελτίωση στην απόδοση, θα δοκιμάσουμε να το πραγματοποιήσουμε και για το 2ο nested loop με τα εξής αποτελέσματα:

Details

Performance estimates for 'calcDistancesHW in main.cpp:16 ...

HW accelerated (Estimated cycles)

251205

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	56	80	70
BRAM	33	60	55
LUT	12882	17600	73.19
FF	10455	35200	29.7

Figure 11: Utilization with double partition moved

Παρατηρούμε ότι για την προσέγγιση αυτή στο loop LOAD_DATA_HW_TMP δεν έχουμε βελτίωση της εκτίμησης του χρόνου, αλλά έχουμε αύξηση των απαιτούμενων πόρων.

- Double unroll

Στην προηγούμενη υλοποίηση, προσθέτουμε στον συνδυασμό των directives το unrolling του τελευταίου loop, έναντι της εντολής για pipelining του:

Details

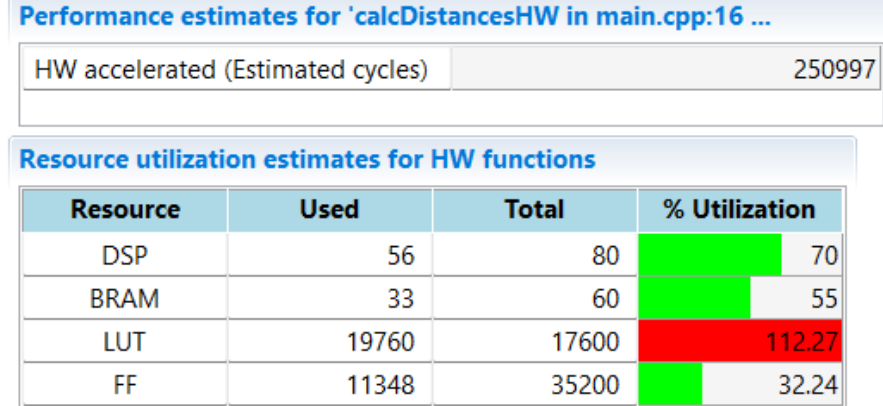


Figure 12: Utilization with double unroll

Μπορούμε να παρατηρήσουμε ότι αν και έχουμε μειωμένο αριθμό κύκλων, βάσει του estimation το zybo δεν έχει τους απαραίτητους πόρους για να τρέξει το design μας (όπως παρουσιάζουμε στην συνέχεια της άσκησης δοκιμάσαμε να το τρέξουμε για να δούμε τα αποτελέσματα).

- Pipeline - Unroll cooperation

Εγκαταλείπεται η παραπάνω προσέγγιση με το unrolling σε αυτήν την φάση για του λόγους που αναλύθηκαν και επιστρέφουμε στο pipelining του τελευταίου loop. Εστιάζουμε την προσέγγισή μας για βελτιστοποίηση στο υπολογιστικά ακριβό loop COMPUTE_DISTS. Δοκιμάζουμε μια προσέγγιση στην οποία κάνουμε pipeline το εξωτερικό loop και το εσωτερικό γίνεται unroll με factor 4, δοκιμάζοντας διάφορες τιμές και παρατηρώντας ότι αυτή είναι η μεγαλύτερη δυνατή τιμή για την οποία βάσει του estimation επαρκούν οι πόροι του zybo και έχουμε:

Details

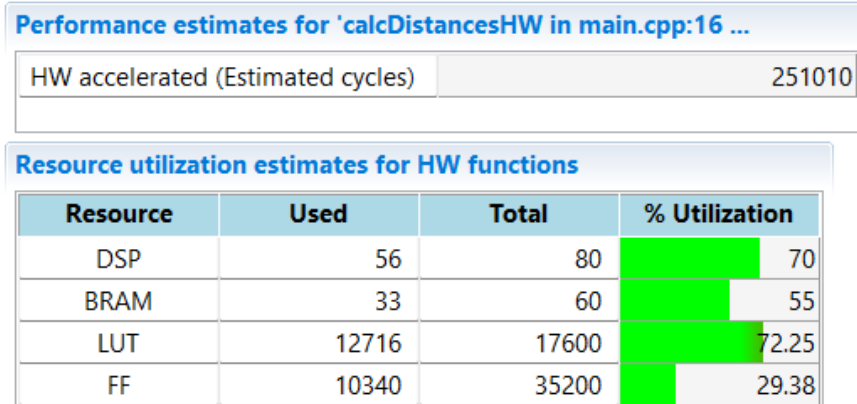


Figure 13: Utilization with pipeline-unroll

Σημειώνουμε ότι για να πετύχουμε με τον καλύτερο τρόπο τα παράλληλα reads-writes κάθε unroll με έναν παράγοντα συνοδεύεται από αντίστοιχο partition του πίνακα με τον ίδιο παράγοντα.

- Partial Unroll

Το τελευταίο optimization που δοκιμάσαμε είναι να κάνουμε unroll το τελευταίο loop όχι πλήρως όπως είχαμε δοκιμάσει και δεν είχαμε τους απαιτούμενους πόρους, αλλάμε παράγοντες 4, 8 και 16 και ενώ παρατηρούμε ότι δεν μειώνεται ο αριθμός κύκλων θα δοκιμάσουμε να τρέξουμε και bitstreams από αυτές τις υλοποιήσεις στο zybo για να δούμε αν όντως στην πραγματικότητα υπάρχει διαφορά.

Σημειώνουμε ότι για όσες αλλαγές και αν δοκιμάσουμε δεν μπορούμε να πετύχουμε ένα πλήρως pipelined design όσο το εσωτερικό loop στο COMPUTE_DISTS δεν είναι πλήρως unrolled, για τους λόγους που αναλύουμε και παραπάνω. Το unroll με factor μεγαλύτερο από 4, όμως, δεν είναι δυνατό σε αυτήν την φάση λόγω των περιορισμών σε πόρους (κρατάμε την σκέψη, γιατί θα είναι χρήσιμη στην άσκηση 3).

Από τα παραπάνω επιλέγουμε να τρέξουμε στο zybo την υλοποίηση με τα unroll που στο estimation δεν φαίνεται να χωράει στο zybo (όνομα double_unroll), την βέλτιστη υλοποίηση μας σε κύκλους, η οποία είναι η αμέσως επόμενη στην ανάλυση πάνω (for_unroll) και την προσέγγιση αυτή με δύο περιπτώσεις όχι πλήρους unroll του τελευταίου loop (ονόματα με βάσει τα factors του unroll unroll8 και unroll32). Παρουσιάζουμε παρακάτω στον πίνακα τα αποτελέσματα σε speedup:

Name	Cycles	Speedup
double_unroll	250963	6.31093
for_unroll	250992	6.32431
unroll4	251006	6.29695
unroll32	251091	6.29605

Παρατηρούμε ότι παρόλο που στα estimations φαίνεται κάποιες από τις υλοποιήσεις να έχουν ίδιο αναμενόμενο αριθμό κύκλων, στην πραγματικότητα υπάρχει διαφορά. Παρατηρούμε ότι η καλύτερη υλοποίηση από άποψη κύκλων είναι αυτή που και βάσει του estimation ήταν αυτή που φαινόταν καλύτερη, η υλοποίηση με τα unroll που δεν φαινόταν να χωράει στο zybo, όμως από ότι φάνηκε η εκτίμηση δεν ήταν σωστή.

Φυσικά, οι υλοποιήσεις μας σε όλες τις περιπτώσεις εμφανίζουν πραγματικό speedup με τιμή μεγαλύτερη του 1, οπότε και παρουσιάζουν ότι υπάρχει πραγματικό κέρδος από την υλοποίηση μέρους του design μας σε hardware στο fpga.

Παρατηρούμε, επιπλέον, ότι αν και οι εκτιμήσεις δεν ταυτίζονται με τον πραγματικό αριθμό κύκλων στο zybo έχουμε αρκετά καλή εκτίμηση των πραγματικών κύκλων σε όλες τις περιπτώσεις.

Σημειώνουμε ότι οι επιλογές μας για τα bitstreams που έτρεξαν στο zybo επηρεάστηκαν από την ανάλυση pareto που παρουσιάζεται στην συνέχεια.

Ερώτημα Δ

Για όλα τα πειράματα που έχουμε τρέξει σε προσομοίωση καταγράφουμε την χρήση σε πόρους και τον αριθμό κύκλων σε ένα αρχείο csv και για κάθε συνδυασμό resource - cycles πραγματοποιούμε ανάλυση pareto με χρήση της βιβλιοθήκης paretoset, την οποία χρησιμοποιήσαμε και στην 1η εργαστηριακή άσκηση. Παρουσιάζουμε παρακάτω τα αποτελέσματα με τις καμπύλες pareto:

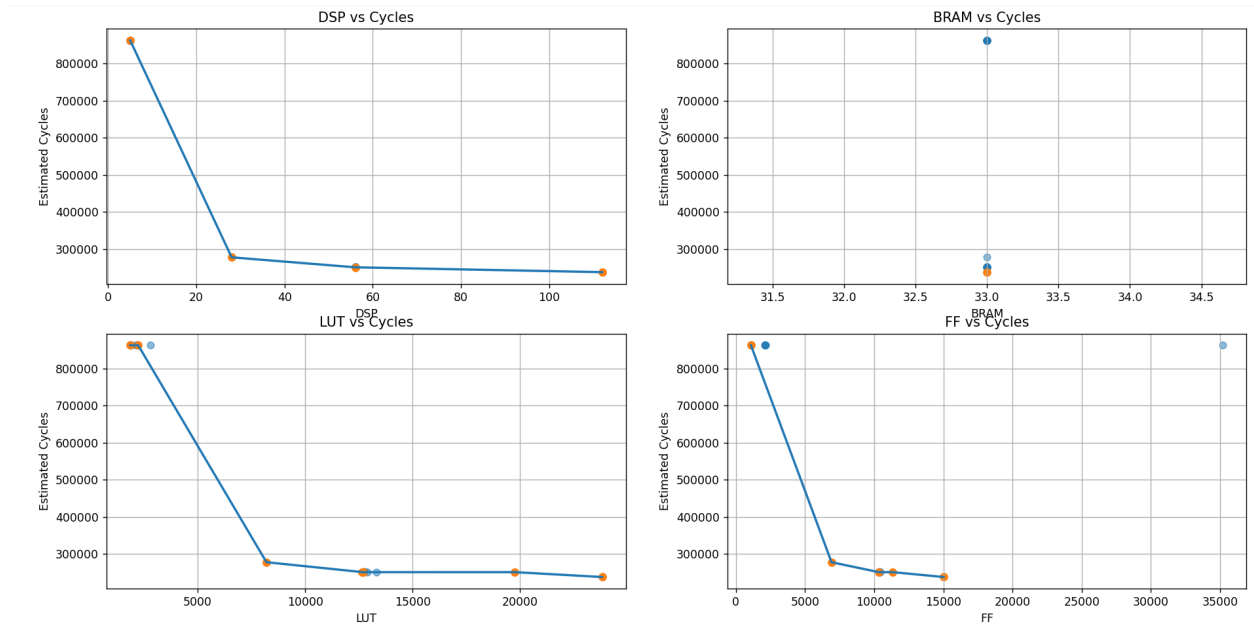


Figure 14: Pareto Fronts

Ένα αρχικό, γενικό, συμπέρασμα που μπορεί να προκύψει από τα pareto fronts που έχουν παραχθεί είναι ότι η αύξηση των πόρων που χρησιμοποιούμε βελτιώνει την απόδοση σε κύκλους, όμως υπάρχει κορεσμός και μετά από ένα σημείο η περαιτέρω αύξηση των χρησιμοποιούμενων πόρων δεν έχει αντίστοιχη βελτιστοποίηση στον αριθμό κύκλων εκτέλεσης. Σημειώνουμε ότι επειδή οι χρόνοι για τα περισσότερα πειράματα που τρέξαμε είναι παρόμοιοι, οι μπλε κουκκίδες που είναι τα σημεία που δεν ανήκουν στο pareto front δεν φαίνονται τόσο καθαρά.

Για τα DSP παρατηρούμε ότι για τις υλοποιήσεις που χρησιμοποιούν περισσότερα από 30 DSP έχουμε βελτιωμένη απόδοση, αλλά η απόδοση δεν κλιμακώνει, το ίδιο καλά, αφού πιθανόν τα designs που χρησιμοποιούν μεγαλύτερο αριθμό δεν θα είναι compute bound.

Για την χρήση BRAMs μπορούμε να παρατηρήσουμε ότι δεν επηρεάζει τον αριθμό κύκλων και για όλες τις υλοποιήσεις μας δεν παρουσιάζεται διαφοροποίηση.

Για το pareto front με τα LUTs, παρατηρούμε από τα προηγούμενα ερωτήματα ότι με unrolls, array partitions και pipelining αυξάνεται ο αριθμός τους. Έτσι, από το διάγραμμα μπορούμε να εξάγουμε συμπεράσματα για την σχέση παραλληλισμού στο design με τον αριθμό κύκλων, με μικρό παραλληλισμό να οδηγεί σε μεγάλο αριθμό κύκλων και μέχρι ένα σημείο περίπου στα 10k - 15k LUTs η αύξηση τους μειώνει σημαντικά τους απαιτούμενους κύκλους, για περισσότερα LUTs, όμως και άρα μεγαλύτερη παραλληλοποίηση δεν παρατηρείται μεγάλη διαφορά σε χρόνο.

Τέλος και για το pareto front με τα FFs παρατηρούμε μια παρόμοια συμπεριφορά με τα LUTs.

Από την παραπάνω ανάλυση, εκτός από το διάγραμμα παράγουμε και csv με τα σημεία του pareto front. Από αυτά τα csv για την επιλογή μας για να τρέξουμε στο zybo επιλέξαμε αυτά που έχουν την βέλτιστη απόδοση σε κάθε περίπτωση (στο ερώτημα Γ). Παρατηρήσαμε πάνω ότι τα designs αυτά έχουν αρκετά παρόμοιο speedup, όμως το utilization τους σε πόρους διαφέρει αρκετά. Το design που επιλέγουμε ως βέλτιστο είναι το design for_unroll (ο κώδικας υποβάλλεται στο αρχείο calcDists1.c), καθώς παρατηρούμε τόσο από τις προσωμιώσεις όσο και από το run στο zybo ότι το speedup του είναι αρκετά ικανοποιητικό και αν και από άποψη αριθμού κύκλων είναι το χειρότερο από αυτά που δοκιμάσαμε, σε κατανάλωση πόρων είναι σημαντικά καλύτερο και εφόσον η διαφορά σε κύκλους είναι 100-150 κύκλους κρίνεται ως το βέλτιστο.

Ουσιαστικά, με αυτόν τον τρόπο δοκιμάσαμε διάφορα designs και βρήκαμε έναν αριθμό κύκλων που φαίνεται να είναι το καλύτερο που μπορούμε να πετύχουμε στην εφαρμογή μας. Στην συνέχεια, επειδή τον χρόνο αυτόν (ή παρεμφερή) πετυχαίνουν αρκετά designs με χρήση των pareto fronts βρίσκουμε τα σημεία που υπάρχει κορεσμός στην μείωση των κύκλων με αύξηση του hardware και επιλέγουμε ένα design που βελτιστοποιεί τον αριθμό κύκλων, χωρίς να καταναλώνει περιττούς πόρους.

Άσκηση 2

Για να μπορούμε να βελτιώσουμε περαιτέρω το design που έχουμε βελτιστοποιήσει στο προηγούμενο ερώτημα παρατηρούμε ότι για τους υπολογισμούς στον κώδικα χρησιμοποιούμε floats, δηλαδή 32 bits για κάθε μεταβλητή, αυτός δεν είναι όμως ο πιο αποδοτικός τρόπος να αποθηκεύουμε τις μεταβλητές για την εφαρμογή.

Με χρήση της βιβλιοθήκης `ap_fixed` θα δημιουργήσουμε έναν custom datatype για το design μας. Αυτό που ουσιαστικά θα πρέπει να καθορίσουμε είναι ο αριθμός των bits για το ακέραιο και για το δεκαδικό μέρος.

Παρατηρούμε ότι οι πιθανές βαθμολογίες για τις ταινίες είναι:

{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0}

και οι εντολές που πραγματοποιούν υπολογισμούς (αριθμητικές πράξεις) μεταξύ των μεταβλητών στην εφαρμογή μας είναι:

```
diff = data_hw_tmp[i][j] - movie_tmp[j];
sum += diff * diff;
...
dists_hw_tmp[i] = sqrt(sum);
```

Η απόλυτη τιμή του `diff` είναι ≤ 5.0 , οπότε το τετράγωνο υπολογίζουμε έχει μέγιστη τιμή 25.0. Η μεταβλητή `sum` προσθέτει τέτοια τετράγωνα και συγκεκριμένα, παρατηρώντας το `loop` που περιλαμβάνει τις παραπάνω εντολές προσθέτει συνολικά 32 τετράγωνα. Η μέγιστη ακέραια τιμή που μπορεί να χρειαστεί να αποθηκεύσουμε, λοιπόν, είναι:

$$25 \cdot 32 = 800$$

Αυτό, όμως, σημαίνει ότι θέλουμε αριθμό bits για το ακέραιο μέρος, με το οποίο να μπορούμε να αναπαραστήσουμε όλους τους θετικούς αριθμούς μέχρι το 800. Για να το πετύχουμε αυτό έχουμε:

$$INT_BITS = 10$$

Η ρίζα φυσικά επειδή μικραίνει το ακέραιο μέρος δεν μπορεί να δημιουργήσει πρόβλημα.

Για το δεκαδικό μέρος τα δεδομένα είναι λίγο πιο περίπλοκα. Τα τετράγωνα τα οποία προστίθενται στο `sum` και άρα και το `sum` μπορούν να αναπαρασταθούν με ακρίβεια 0.25. Η ρίζα, όμως, σε αυτήν την περίπτωση δημιουργεί το πρόβλημα ότι ακόμα και αν έχουμε ακέραιο, μετά την ρίζα ίσως χρειαστούμε αρκετά δεκαδικά ψηφία για την αναπαράσταση της μεταβλητής. Ενδεικτικά αναφέρουμε ότι αν θέλουμε ακρίβεια 0.0001 στην αναπαράσταση μας χρειαζόμαστε 14 bits δεκαδικού μέρους και για 0.001 10 bits.

Φυσικά, επειδή η αναπαράσταση των μεταβλητών επηρεάζει άμεσα το design μας, πρέπει να επιλέξουμε μια τιμή η οποία να μας παρέχει μια ακρίβεια που θα εξασφαλίζει ότι η εφαρμογή μας θα έχει σωστά αποτελέσματα, αλλά χωρίς να χρησιμοποιήσουμε περιττά bits. Επειδή παρατηρούμε από τα προηγούμενα runs ότι οι διαφορές ανάμεσα στις προτάσεις έχουν ελάχιστη τιμή απόστασης 0.0911, οπότε έχουμε για τον αριθμό bits:

$$step = 2^{-DEC_BITS} \leq 0.0911$$

$$DEC_BITS \geq 3.46$$

Οπότε, για τις τιμές που εμφανίζονται στα αποτελέσματα μας αρκούν 4 bits. Επειδή, όμως δεν ξέρουμε αν αυτό θα αρκεί και για τα υπόλοιπα δεδομένα του dataset θα επιλέξουμε $DEC_BITS = 5bits$ αρχικά και ανάλογα με τα αποτελέσματα για χρήση του hardware ή λόγω πιθανών σφαλμάτων που θα παρατηρηθούν ίσως αυξήσουμε την τιμή σε 6 ή 7 bits.

Με την επιλογή των 5 bits έχουμε ακρίβεια 0.03125, η οποία φαίνεται να μας αρκεί στην περίπτωση της εφαρμογής μας. Σε αυτό το σημείο επισημαίνουμε ότι οι σχέψεις μας για τον αριθμό των bits που θα χρησιμοποιηθούν στο δεκαδικό μέρος κυμάνθηκαν από τα 2 bits που είναι η ακραία περίπτωση για την βέλτιστη χρήση hardware, αλλά με πιθανό κόστος σε ακρίβεια του αλγορίθμου, μέχρι τα 7 bits, για τα οποία θα έχουμε αρκετή ασφάλεια για την ακρίβεια του αλγορίθμου, αλλά η εξοικονόμηση σε πόρους δεν θα ήταν τόσο κοντά στην ιδανική.

Δοκιμάσαμε, ωστόσο με 2 bits δεκαδικό μέρος το performance estimation και με 2 bits δεκαδικό μέρος και αναλύουμε στην συνέχεια.

Δοκιμάζουμε αρχικά το performance estimation με τα directives που είχαμε στην υλοποίηση που είχαμε θεωρήσει βέλτιστη στην προηγούμενη άσκηση:

Details

Performance estimates for 'calcDistancesHW in main.cpp:16 ...

HW accelerated (Estimated cycles)	250256
-----------------------------------	--------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	32	80	40
BRAM	17	60	28.33
LUT	5198	17600	29.53
FF	2817	35200	8

Figure 15: Custom Datatypes Unoptimized - Utilization

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- LOAD_DATA_HW_TMP_L	32771	32771	5	1	1	32768	yes
- COMPUTE_DISTS	4123	4123	32	4	1	1024	yes
- WRITE_DISTS	1024	1024	2	1	1	1024	yes

Figure 16: Custom Datatypes Unoptimized - Loops

Παρατηρούμε μια μικρή βελτίωση στην ταχύτητα εκτέλεσης της εφαρμογής μας σε αριθμό κύκλων. Ωστόσο, η αλλαγή από float σε custom datatypes δεν έχει ως μεγαλύτερο άμεσο όφελος την βελτίωση της επίδοσης, αλλά την βελτίωση στον τρόπο χρήσης των πόρων του fpga. Παρατηρούμε ότι η χρήση των DSP και BRAM πέφτει σχεδόν στο μισό, ενώ η μείωση στην χρήση των LUTs και FFs είναι αντίστοιχα κάτω από το 1/2 και 1/4. Αυτό σημαίνει, δηλαδή, ότι με την ίδια εφαρμογή και μοναδική αλλαγή την δημιουργία ενός εξειδικευμένου datatype για αυτήν μπορούμε να μειώσουμε το hardware που απαιτείται σε πολύ σημαντικό βαθμό.

Όπως αναφέρθηκε και πάνω έγινε και δοκιμή με 2 bits δεκαδικό μέρος, με ακρίβεια 0.25 η οποία μας καλύπτει σε όλα τα κομμάτια της εφαρμογής εκτός από το κομμάτι της ρίζας, από το οποίο τελικά θα παραχθούν οι αποστάσεις και άρα έχει σημαντική συμβολή στην επίδοση της εφαρμογής μας:

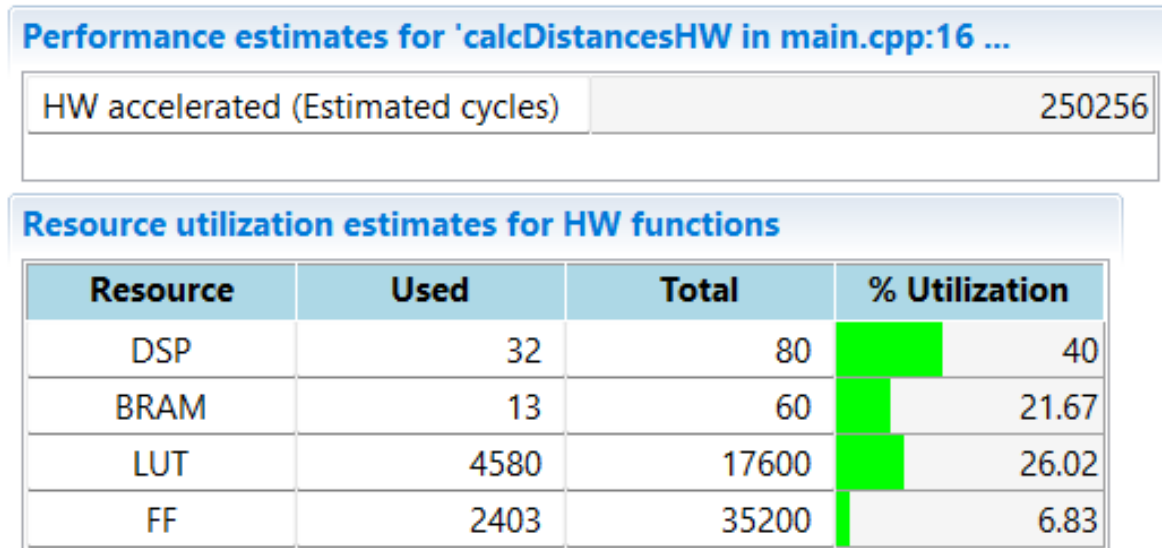


Figure 17: Custom Datatypes Unoptimized - 2 decimal bits

Παρατηρούμε ότι δεν υπάρχει τόσο έντονη διαφορά στην εξοικονόμηση πόρων όση υπάρχει από το datatype float στο custom datatype με 10 bits ακέραιο μέρος και 5 δεκαδικό. Κρίνεται ως καλύτερη η επιλογή των 5 bits δεκαδικού μέρους, λοιπόν, λόγω του πιθανού κόστους στην ακρίβεια της εφαρμογής μας σε αυτήν την φάση.

Δοκιμάζουμε να τρέξουμε και στο zybo την έκδοση αυτή της εφαρμογής μας:

```

Recommendation system start to make inference
...
Recommendations for movie with id 0:
0. Jurassic Park (1993), with distance of 10.7121
1. Fish Called Wanda A (1988), with distance of 11.0793
2. Back to the Future (1985), with distance of 11.6404
3. Star Wars: Episode VI - Return of the Jedi (1983), with distance of 11.8849
4. Lion King The (1994), with distance of 12.0623
5. Raising Arizona (1987), with distance of 12.1552
6. Wizard of Oz The (1939), with distance of 12.1861
7. Batman (1989), with distance of 12.2066
8. Princess Bride The (1987), with distance of 12.3085
9. E.T. the Extra-Terrestrial (1982), with distance of 12.3996

Hardware cycles : 250301
Software cycles : 1587069
Speedup : 6.34064
Correct = 1024, Score = 1.000000
sh-4.3# █

```

Figure 18: Custom Datatypes Unoptimized zybo run

Παρατηρούμε ότι πάλι η εφαρμογή μας πραγματοποιεί σωστές προτάσεις και ο αριθμός κύκλων είναι λίγο μειωμένος όπως ήταν αναμενόμενο και από το performance estimation.

Άσκηση 3

Όπως έχουμε αναφέρει πάνω η αλλαγή σε custom datatype έχει μειώσει το χρησιμοποιούμενο hardware σημαντικά. Αυτό σημαίνει ότι η ήδη υπάρχουσα optimized εφαρμογή μπορεί να τρέχει με λιγότερο διαθέσιμο hardware. Παράλληλα, όμως μας δίνεται ευκαιρία να πραγματοποιήσουμε optimizations τα οποία δεν μπορούσαμε να πραγματοποιήσουμε στην προηγούμενη έκδοση της εφαρμογής λόγω των διαθέσιμων πόρων του fpga.

Η 1η αλλαγή που θα πραγματοποιήσουμε, φυσικά, είναι να πραγματοποιήσουμε πλήρες unroll του εσωτερικού loop του COMPUTE_DISTS, ώστε να μπορέσουμε να πετύχουμε πλήρως pipelined design. Σημειώνουμε ότι συνοδεύουμε την αλλαγή αυτή με αύξηση του factor του array partition για τον πίνακα hw_data_tmp σε 32 επίσης. Παρατηρούμε για αυτήν την αλλαγή στα αποτελέσματα παρακάτω ότι με το εσωτερικό loop πλήρως unrolled το design μας μπορεί να είναι πλήρως pipelined, αφού και το loop COMPUTE_DISTS πλέον έχει $\Pi = 1$. Η αλλαγή αυτή κάνει και την μεγαλύτερη διαφορά σε επίδοση από αυτές που δοκιμάζουμε σε αυτήν την άσκηση όπως είναι λογικό.

Η 2η αλλαγή είναι να κάνουμε full unroll το τελευταίο loop. Παρατηρούμε για αυτήν την αλλαγή ότι το κέρδος σε κύκλους είναι μικρό, αλλά εφόσον βελτιώνει, έστω και λίγο την απόδοση επιλέγουμε να την κρατήσουμε.

Σημειώνεται σε αυτό το σημείο ότι όσες αλλαγές δοκιμάσαμε σε array partitions με διαφορετικό τρόπο από αυτόν που έχουμε δεν φαίνεται να βελτιώνει την επίδοση, κρατώντας παράλληλα την εφαρμογή μας εντός των περιορισμών του hardware. Δοκιμάσαμε, επίσης, να μετατρέψουμε τον πίνακα data_hw_tmp σε 1D, με τις αντίστοιχες απαραίτητες αλλαγές στον κώδικα και το αντίστοιχο array partition φυσικά, ωστόσο, δεν παρατηρήθηκε βελτίωση της επίδοσης στο performance estimation και η χρήση LUTs και FFs ήταν λίγο αυξημένη και έτσι εγκαταλήφθηκε αυτή η προσέγγιση.

Details

Performance estimates for 'calcDistancesHW in main.cpp:16 ...

HW accelerated (Estimated cycles)	230275
-----------------------------------	--------

Resource utilization estimates for HW functions

Resource	Used	Total	% Utilization
DSP	32	80	40
BRAM	17	60	28.33
LUT	12169	17600	69.14
FF	3719	35200	10.57

Figure 19: Custom Datatypes Optimized - Utilization

Loop

Loop Name	Latency		Iteration Latency	Initiation Interval		Trip Count	Pipelined
	min	max		achieved	target		
- LOAD_DATA_HW_TMP_L	32771	32771	5	1	1	32768	yes
- COMPUTE_DISTS	1054	1054	32	1	1	1024	yes

Figure 20: Custom Datatypes Optimized - Loops

Συνολικά, παρατηρούμε ότι αυξάνεται η χρήση των πόρων σε σχέση με την unoptimized έκδοση με τους custom datatypes, όμως έχουμε και σημαντικό κέρδος σε estimated performance. Ουσιαστικά, λοιπόν, δημιουργήσαμε έναν custom datatype για να εξοικονομήσουμε hardware, το οποίο στην συνέχεια να μπορούμε να αξιοποιήσουμε ώστε να πετύχουμε ένα ακόμη ταχύτερο design.

Παρακάτω παρουσιάζουμε και αυτήν την υλοποίηση της εφαρμογής σε run στο zybo:

```

Recommendation system start to make inference
...
Recommendations for movie with id 0:
0. Jurassic Park (1993), with distance of 10.7121
1. Fish Called Wanda A (1988), with distance of 11.0793
2. Back to the Future (1985), with distance of 11.6404
3. Star Wars: Episode VI - Return of the Jedi (1983), with distance of 11.8849
4. Lion King The (1994), with distance of 12.0623
5. Raising Arizona (1987), with distance of 12.1552
6. Wizard of Oz The (1939), with distance of 12.1861
7. Batman (1989), with distance of 12.2066
8. Princess Bride The (1987), with distance of 12.3085
9. E.T. the Extra-Terrestrial (1982), with distance of 12.3996

Hardware cycles : 230179
Software cycles : 1589406
Speedup          : 6.90509
Correct = 1024, Score = 1.000000
sh-4.3# █

```

Figure 21: Custom Datatypes Optimized - zybo run

Παρατηρούμε ότι έχουμε το καλύτερο speedup που έχουμε πετύχει στο σύνολο των ασκήσεων στο εργαστήριο αυτό με **6.90509**.

Στο τέλος της 1ης άσκησης πετύχαμε ένα βέλτιστο speedup περίπου 6.32, αυτό όμως δεν μπορούσαμε να το βελτιστοποιήσουμε περισσότερο, καθώς δεν θα είχαμε τους απαραίτητους πόρους στο zybo. Η επιλογή που κάνουμε για να καταφέρουμε να έχουμε επιπλέον hardware στο ίδιο fpga είναι να δημιουργήσουμε ένα custom data type, το οποίο να μας βοηθάει να εξοικονομήσουμε hardware. Έτσι, μπορούμε να πειραματιστούμε με πιο έντονα optimizations και να πετύχουμε ένα πλήρως pipelined design, χωρίς να εμφανιστεί πρόβλημα utilization των πόρων του fpga.

Σημειώσεις για τους κώδικες

Σημειώνουμε ότι στο zip που υποβάλλεται έχουμε 3 φακέλους code_ask.i με περιεχόμενα calcDist.i.cpp, h, όπου i είναι ο αριθμός της άσκησης κάθε φορά και εκεί παρουσιάζουμε τον τελικό (βέλτιστο) κώδικα για κάθε άσκηση.